

Virtual-Air-Mouse

<https://github.com/Ma1cOS/Virtual-Air-Mouse>



Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών

**Δημήτριος
Παρασκευόπουλος**

Προπτυχιακός Φοιτητής

AM: 21390314

Email: ice21390314@uniwa.gr

Github: [jimpar1](https://github.com/jimpar1)

**Μάκο
Σωτήρης**

Προπτυχιακός Φοιτητής

AM: 20390136

Email: ice20390136@uniwa.gr

Github: [Ma1cOS](https://github.com/Ma1cOS)

**Τραχίλης
Θωμάς**

Προπτυχιακός Φοιτητής

AM: 22390225

Email: ice22390225@uniwa.gr

Github: [Ttrahilis](https://github.com/Ttrahilis)

1. Περίληψη

Το έγγραφο αυτό αποτελεί την γραπτή τεκμηρίωση της Απαλλακτικής Εργασίας του μαθήματος Όρασης Υπολογιστών για το Ακαδημαϊκό έτος 2025-2026.

1.1 Σκοπός

Δημιουργία ενός εικονικού ποντικιού που ελέγχεται αποκλειστικά μέσω κάμερας και χειρονομιών του χεριού. Ο χρήστης κουνάει το χέρι του μπροστά από την κάμερα και ο κέρσορας ακολουθεί σε πραγματικό χρόνο. Τα clicks γίνονται με pinch gestures: ο αντίχειρας ακουμπάει διαφορετικά δάχτυλα για left click, right click και middle click. Το σύστημα είναι σχεδιασμένο για linux distributions με display server το λογισμικό Wayland.

1.2 Μεθοδολογία

Το σύστημα δουλεύει σε βήματα. Η κάμερα τροφοδοτεί το MediaPipe, το οποίο εντοπίζει 21 σημεία πάνω στο χέρι. Μετά, ένα adaptive EMA φίλτρο εξομαλύνει τις συντεταγμένες αυτές και υπολογίζονται τα διανύσματα dx και dy με βάση το τωρινό και το προηγούμενο frame. Έπειτα, τα διανύσματα στέλνονται στον kernel, ως REL_X/REL_Y events με /dev/uinput. Τα pinch gestures τα διαχειρίζεται μια custom μηχανή καταστάσεων.

1.3 Αποτελέσματα

Το σύστημα επιτυγχάνει ομαλή κίνηση κέρσορα χωρίς θόρυβο, χάρη στο συνδυασμό EMA φιλτραρίσματος και sub-pixel accumulation, αλλά και την μη αντιστοίχιση σημείων κάμερας-οθόνης, λόγω της χρήσης REL έναντι ABS events.

Τα pinch gestures ανταποκρίνονται αξιόπιστα με ελάχιστα false positives. Το εικονικό ποντίκι συνυπάρχει με το φυσικό ποντίκι χωρίς παρεμβολές.

1.4 Το πρόβλημα

Σε περιβάλλοντα Wayland, δεν επιτρέπεται σε μια εφαρμογή να μετακινήσει τον κέρσορα ή να προσομοιώσει clicks, για λόγους ασφαλείας [6]. Αυτό σημαίνει ότι όλες οι κλασικές βιβλιοθήκες αυτοματισμού αποτυγχάνουν. **Η λύση είναι απευθείας events στον kernel**, μιλάμε απευθείας στον Linux kernel.

Το **uinput** είναι ένα kernel module που επιτρέπει σε userspace εφαρμογές να δημιουργούν εικονικές συσκευές εισόδου. Η Python βιβλιοθήκη **evdev** προσφέρει ένα απλό API πάνω από το uinput [3], [4]. Δημιουργούμε μια εικονική συσκευή ποντικιού και της στέλνουμε REL_X, REL_Y events για κίνηση και BTN_LEFT, BTN_RIGHT, BTN_MIDDLE για clicks.

Ο kernel τα παραδίδει στον compositor σαν να προέρχονται από κανονικό ποντίκι.

2. Θεωρητικό Υπόβαθρο

2.1 MediaPipe: hand landmark detection

Το MediaPipe είναι μια βιβλιοθήκη μηχανικής μάθησης της Google για real-time επεξεργασία ροών δεδομένων. Το hand landmark model εντοπίζει **21 σημεία** (landmarks) πάνω στο χέρι: 4 σημεία ανά δάχτυλο συν τη βάση της παλάμης. Κάθε landmark έχει συντεταγμένες (x,y,z) κανονικοποιημένες στο [0,1]. [1], [2]

Πώς οι συντεταγμένες μεταφράζονται σε pixel της κάμερας;

Η αντιστοίχιση των συντεταγμένων σε πραγματικά pixels γίνεται πολλαπλασιάζοντας την κανονικοποιημένη τιμή με την αντιστοιχη διάσταση της ανάλυσης. Τελικά παίρνουμε το ακέραιο μέρος. Αγνοούμε το z, καθώς η κίνηση του κέρσορα γίνεται σε διδιάστατο χώρο.

Για παράδειγμα

Έστω τις αρχικές συντεταγμένες $x = 0.318$, $y = 0.5$ και διαστάσεις κάμερας **640x480**.

$raw_x = \text{int}(0.318 \cdot W) = \text{int}(0.318 \cdot 640) = \text{int}(203.52) = 203$

$raw_y = \text{int}(0.500 \cdot H) = \text{int}(0.500 \cdot 480) = \text{int}(240.00) = 240$

Άρα το pixel που αντιστοιχεί είναι το (203,240).

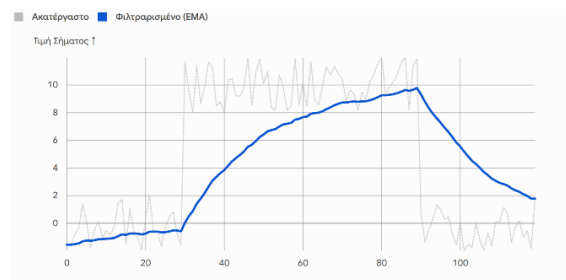
2.2 EMA: Exponential Moving Average

Το EMA είναι ένα αναδρομικό φίλτρο που υπολογίζει έναν σταθμισμένο μέσο όρο μεταξύ της τρέχουσας μέτρησης και της προηγούμενης εξομαλυνμένης τιμής

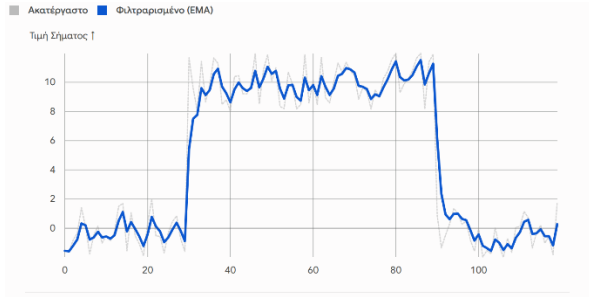
Τύπος

$$S_t = a \cdot x_t + (1 - a) \cdot S_{t-1}$$

S_t	Νέα εξομαλυνμένη θέση
x_t	Raw θέση από το MediaPipe
a	Συντελεστής EMA
S_{t-1}	Προηγούμενη εξομαλυνμένη θέση



Εικόνα 1: Μικρό a (π.χ. 0.05) σημαίνει αργή απόκριση, μεγάλη εξομάλυνση.



Εικόνα 2: Μεγάλο α (π.χ. 0.50) σημαίνει γρήγορη απόκριση, λιγότερη εξομάλυνση.

2.3 Stability locking

Το φίλτρο μας είναι **προσαρμοστικό**: το α αλλάζει ανάλογα με την ταχύτητα κίνησης του χεριού. Αυτό συμβαίνει όταν d (ευκλείδια απόσταση) μεταξύ raw και smoothed θέσης είναι **μικρότερη από 15 pixels**, το α μειώνεται στο 10% της κανονικής του τιμής.

Για παράδειγμα

$$\text{Έστω } \alpha = \begin{cases} 0.012, & \text{Αν } d < 15 \\ 0.120, & \text{Άλλο} \end{cases}$$

Περίπτωση 1^η: Έστω ότι ο χρήστης κουνάει το χέρι του από το pixel 500 σε μια νέα θέση, στο pixel 550.

$$\begin{aligned} s_t &= 0.12 \cdot x_t + 0.88 \cdot s_{t-1} \Leftrightarrow \\ s_t &= 0.12 \cdot 550 + 0.88 \cdot 500 \Leftrightarrow \\ s_t &= 66 + 440 = 506 \end{aligned}$$

Συνεπώς, η νέα εξομαλυμένη θέση είναι 506.

Περίπτωση 2^η: Έστω ότι ο χρήστης κρατάει το χέρι του ακίνητο, αλλά λόγω θορύβου της κάμερας, ή του νευρωνικού δικτύου το MediaPipe βγάζει λανθασμένα ότι η νέα θέση είναι στο 505 pixel, με προηγούμενη θέση το 500 pixel.

$$\begin{aligned} s_t &= 0.012 \cdot x_t + 0.988 \cdot s_{t-1} \Leftrightarrow \\ s_t &= 0.012 \cdot 505 + 0.988 \cdot 500 \Leftrightarrow \\ s_t &= 6.06 + 494 = 500.06 \Leftrightarrow \\ \text{int}(500.06) &= 500 \end{aligned}$$

Συνεπώς, η νέα εξομαλυμένη θέση είναι η ίδια με την προηγούμενη.

2.4 Υπολογισμός σχετικής κίνησης

Το σύστημα απαιτεί την εισαγωγή γεγονότων σχετικής μετατόπισης REL_X, REL_Y για την κίνηση του κέρσορα, και όχι απόλυτων συντεταγμένων. Συμπερασματικά, μετά το φιλτράρισμα EMA, οι απόλυτες εξομαλυμένες συντεταγμένες S_x, S_y πρέπει να μετατραπούν σε ακέραια διανύσματα σχετικής κίνησης d_x, d_y .

Τύπος

$$\begin{aligned} \Delta x_{cam} &= s_{x,t} - s_{x,t-1} \\ \Delta y_{cam} &= s_{y,t} - s_{y,t-1} \end{aligned}$$

$\Delta x_{cam}, \Delta y_{cam}$	Καθαρή μετατόπιση σε pixel κάμερας
$s_{x,t}, s_{y,t}$	Τρέχουσα εξομαλυμένη θέση
$s_{x,t-1}, s_{y,t-1}$	προηγούμενη εξομαλυμένη θέση

2.5 Κλιμάκωση και Συσσώρευση

Η μετατόπιση πολλαπλασιάζεται με έναν συντελεστή κλιμάκωσης για τη μετάβαση από το σύστημα συντεταγμένων της κάμερας στην ταχύτητα του sensitivity. Επειδή το ποντίκι δέχεται μόνο ακέραιες τιμές, η απλή στρογγυλοποίηση θα μηδένιζε τις πολύ αργές κινήσεις (π.χ. $[0.8] = 0$).

Για να λυθεί αυτό, τα δεκαδικά υπόλοιπα προστίθενται σε συσσωρευτές (accumulators) [7]

Τελικοί τύποι κίνησης

$$A_x = A_{x,prev} + (\Delta x_{cam} \cdot S)$$

A_x	Νέα συνολική κίνηση, ακέραιο συν δεκαδικό μέρος
$A_{x,prev}$	Δεκαδικό υπόλοιπο από το προηγούμενο frame
S	Συντελεστής κλιμάκωσης

Η τελική κίνηση που αποστέλλεται στο λειτουργικό είναι το ακέραιο μέρος του συσσωρευτή, ενώ το νέο δεκαδικό υπόλοιπο κρατείται για τον επόμενο κύκλο.

$$dx = \lfloor A_x \rfloor$$

$$A_{x,new} = A_x - d_x$$

dx	Τελική ακέραια κίνηση(αποστολή στους kernel)
$A_{x,new}$	Νέο δεκαδικό υπόλοιπο για το επόμενο frame

3. Αντιστοίχιση χειρονομιών σε ενέργειες Συστήματος

Για να υλοποιήσουμε τα gestures, αποφασίσαμε να ορίσουμε αρχικά τον αντίχειρα ως base finger και ύστερα τον δείκτη, τον μέσο και τον παράμεσο ως click fingers, με την παραδοχή ότι κατά τον συνδυασμό των δύο τύπων (base & click) μπορεί να εκτελεστεί (στιγμιαία ή παρατεταμένα) μια ενέργεια από αυτές που ορίσαμε (left, right & middle click αντίστοιχα).

3.1 Εύρεση της εκάστοτε κατάστασης χειρονομίας ή της απουσίας της

Σε κάθε επεξεργασμένο καρέ, υπολογίζουμε την ευκλείδια απόσταση του base finger από κάθε click finger. Μόλις βρούμε το click finger με την ελάχιστη ευκλείδια απόσταση, ελέγχουμε αν αυτή είναι μικρότερη από το κατώφλι απόστασης που έχουμε ορίσει.

Σε περίπτωση που η απόσταση είναι μεγαλύτερη από το κατώφλι, θεωρούμε ότι δεν υπάρχει σύνδεση με τον αντίχειρα και ενημερώνουμε ότι η τωρινή κατάσταση χειρονομίας είναι άδεια (κρατώντας επίσης την αμέσως προηγούμενη κατάσταση χειρονομίας ως την παλιά κατάσταση).

Διαφορετικά, εφόσον είμαστε εντός του κατωφλίου, καταγράφουμε την καινούρια κατάσταση χειρονομίας (η οποία θα είναι είτε η LEFT είτε η RIGHT είτε η MIDDLE), δηλαδή η αντίστοιχη χειρονομία (gesture) που έχουμε αναθέσει από το config για το συγκεκριμένο δάχτυλο.

3.2 Διαχείριση των gesture states από την μηχανή καταστάσεων

Είτε έχουμε κατάσταση χειρονομίας είτε απουσία της, και στις δύο περιπτώσεις ενημερώνεται η μηχανή καταστάσεων των χειρονομιών, όπου υπολογίζεται αν έχουμε είτε σύνδεση είτε αποσύνδεση των δακτύλων σε κάθε καρέ, με βάση την τωρινή και την προηγούμενη κατάσταση χειρονομίας.

Αποσύνδεση των δακτύλων έχουμε όταν η καινούρια κατάσταση είναι η άδεια (NONE) και οι παλιές καταστάσεις είναι οι LEFT, RIGHT, MIDDLE, ενώ σύνδεση των δακτύλων έχουμε όταν η νέα κατάσταση χειρονομίας δεν είναι άδεια (NONE).

3.2.1 Σύνδεση δακτύλων

Όταν γίνεται σύνδεση ενός από τα δάκτυλα με τον αντίχειρα, αν η τωρινή κατάσταση χειρονομίας διαφέρει από την προηγούμενη, καταγράφεται η χρονική στιγμή της έναρξης σύνδεσης με τον αντίχειρα. Ύστερα, για κάθε επόμενο καρέ όπου τα ίδια δάκτυλα παραμένουν ενωμένα, δηλαδή έχουμε την ίδια κατάσταση χειρονομίας, υπολογίζεται ο χρόνος της σύνδεσης με τον αντίχειρα. Αν αυτός υπερβαίνει το κατώφλι παρατεταμένης χειρονομίας, τότε ενεργοποιούμε παρατεταμένα την αντίστοιχη λειτουργία (αριστερό δεξί ή μεσαίο κλικ) της εικονικής συσκευής.

3.2.2 Αποσύνδεση δακτύλων

Όταν γίνεται αποσύνδεση των δακτύλων, καταγράφουμε τον χρόνο της αποσύνδεσης και υπολογίζουμε την διάρκεια. Αν η διάρκεια σύνδεσης με τον αντίχειρα είναι μικρότερη από το κατώφλι που ορίσαμε για το στιγμιαίο πάτημα, αγνοούμε το σήμα και δεν στέλνουμε τίποτα στο λειτουργικό, ώστε να μην έχουμε εσφαλμένες ενεργοποιήσεις ή ανεπιθύμητο θόρυβο από μικρές, τυχαίες κινήσεις των δακτύλων. Αν όμως η διάρκεια σύνδεσης υπερβαίνει το κατώφλι του απλού πατήματος και είναι μικρότερη από το κατώφλι του παρατεταμένου πατήματος, στέλνουμε σήμα στο λειτουργικό να πραγματοποιήσει το στιγμιαίο κλικ. Επιπλέον, κατά την απελευθέρωση των δακτύλων πραγματοποιείται σε κάθε

περίπτωση και η απελευθέρωση του παρατεταμένου κλικ αν αυτό ήταν ενεργοποιημένο.

4. Αρχιτεκτονική Συστήματος

4.1 Βελτιστοποίηση για καθημερινή χρήση

Το σύστημα είναι υλοποιημένο και καταναμεμένο σε τρία threads. Το κύριο νήμα τρέχει όλες τις συναρτήσεις που καλούνται στο app/main.py και κάνει ανίχνευση συντεταγμένων μέσω της εισόδου εικόνας (χρησιμοποιώντας νευρωνικό δίκτυο), εξομάλυνση των συντεταγμένων, αναγνώριση των gestures, και αποστολή των εντολών του ποντικιού στο λειτουργικό. Τα παραπάνω γίνονται σειριακά, ένα frame τη φορά.

Όμως, η καταγραφή της εικόνας του κάθε frame και η διάθεσή της στο κύριο νήμα γίνεται από ένα παράλληλο thread, έτσι ώστε να εξοικονομηθεί χρόνος υπολογισμού και να διασφαλιστεί η ομαλότητα της κίνησης του χρήστη.

Επιπλέον, για την επίτευξη του προαναφερθέντος στόχου, επιλέχθηκε η εμφάνιση του γραφικού περιβάλλοντος του χρήστη να γίνει και αυτή σε ξεχωριστό παράλληλο νήμα, διότι είναι επίσης αρκετά απαιτητική. Περιέχει την συνεχή και ενημερωμένη εμφάνιση της εικόνας του χρήστη σε ένα γραφικό παράθυρο, έχοντας αυτή την εικόνα συνεχώς επεξεργασμένη για να εμφανίζονται οι ακμές του χεριού καθώς και οι κόμβοι των άκρων του όπως εμφανίζονται και στιγμιαία και ομαλά ύστερα από την εξομάλυνση. Επιπλέον, πέρα από την οπτική ανατροφοδότησή του χρήστη (η οποία τον πληροφορεί άμεσα και εύκολα για την επιτυχία της ανίχνευσης που γίνεται), απαιτείται η συνεχής αναμονή για πληκτρισμούς είτε της ενεργοποίησης του έξυπνου ποντικιού, είτε της έξοδου από το πρόγραμμα, η οποία αν δεν γινόταν σε παράλληλο νήμα θα καθυστερούσε ιδιαίτερα τους σειριακούς υπολογισμούς του κύριου νήματος.

Ανακεφαλαιώνοντας, για να μπορέσει το έξυπνο ποντίκι να εκτελεστεί σε πραγματικό χρόνο, με ακρίβεια, ομαλότητα και να βολεύει στην καθημερινή χρήση, πέρα από το κύριο νήμα χρειάζονται και δύο daemon threads τα οποία κρατάνε την κάμερα και το GUI ζωντανά χωρίς να μπλοκάρουν τον βασικό βρόγχο εκτέλεσης.

4.2 Βασικός βρόχος

Κάθε frame περνάει από τέσσερα στάδια, το καθένα σε δικό του module.

4.2.1 Module 1: vision

Κάθε frame περνάει από (BGR σε RGB [5], detect_for_video, φιλτράρισμα) και μετά από (Κανονικοποιημένες τιμές σε pixel, 21 σημεία ενδιαφέροντος σαν λίστα).

Η σειρά είναι αυστηρή, όπου πρώτα Εντοπίζεται το χέρι, μετά τα σημεία ενδιαφέροντος στο ίδιο frame. Το module είναι αυτόνομο, το μόνο που χρειάζεται είναι μια εικόνα BGR.

4.2.2 Module 2: smoothing

Συντονίζει 5 ανεξάρτητα φίλτρα. Ο κέρσορας με αναγνωριστικό αριθμό σημείου ενδιαφέροντος 0, έχει ανεξάρτητο φίλτρο από τα υπόλοιπα σημεία ενδιαφέροντος. Το φίλτρο του κέρσορα διαθέτει το κατώφλι το οποίο μειώνει το α το φίλτρου στο 10%.

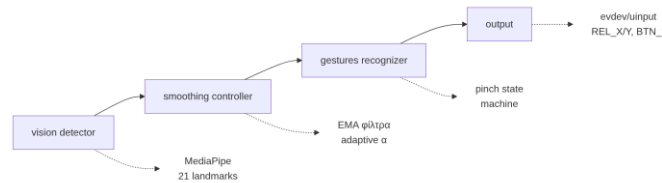
4.2.3 Module 3: gestures

Συντονίζει με μηχανή καταστάσεων, ποιο δάχτυλο ακουμπάει το σημείο ενδιαφέροντος που αντιστοιχεί στον αντίχειρα. Για να θεωρηθεί ότι κάποιο δάχτυλο να πάει στον αντίχειρα έχει τεθεί ως παράμετρος η ελάχιστη απόσταση 40 μονάδων. Οι καταστάσεις που μπορούν να έχουν τα δάκτυλα που ακουμπάνε τον αντίχειρα

είναι το click, το hold, drag. Γιατί κάθε κατάσταση οριοθετείτε ένα εύρος χρόνου. Κάτω από την ελάχιστη τιμή του εύρους, η κατάσταση αγνοείται.

4.2.4 Module 4: output

Δημιουργείται μια εικονική συσκευή uinput, όπου υποστηρίζει σχετική κίνηση (REL_X, REL_Y) και της προαναφερθέντες κατάστασης (BTN_LEFT, BTN_RIGHT, BTN_MIDDLE) [3], [4]. Παρέχει την τελική συνάρτηση κίνησης, η οποία μεταφέρει το τελικό αποτέλεσμα απευθείας στον πυρήνα ώστε και αυτός με τη σειρά του να εκτελέσει την κίνηση μέσω του display server.



Εικόνα 3: Pipeline βρόχου.

5. Ρυθμός δειγματοληψίας

Το σύστημα ενεργοποίησε τη δυνατότητα δειγματοληψίας πέραν από τον ρυθμό δειγματοληψίας της κάμερας. Όμως, μεγαλύτερη δειγματοληψία από τη συχνότητα της πηγής οδηγεί σε διπλότυπα στιγμιότυπα. Παρ' όλα αυτά, το φίλτρο εξομάλυνσης EMA ήδη λύνει αυτό το πρόβλημα: το τωρινό στιγμιότυπο δεν συγκρίνεται με τη raw τιμή αλλά με το προηγούμενο εξομαλυμένο, το οποίο έχει μετατοπιστεί μόνο κατά α τοις εκατό. Στην πράξη, όταν δύο διαδοχικά frames είναι πανομοιότυπα, μειώνεται μόνο ο ρυθμός μεταβολής (dx , dy).

6. Αναφορές

- [1] Google LLC. 2024. Hand landmarks detection guide. MediaPipe. Retrieved from https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- [2] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. 2020. MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214. Retrieved from <https://arxiv.org/abs/2006.10214>
- [3] Linux Kernel Developers. 2023. uinput: User-level input driver. Linux Kernel Documentation. Retrieved from <https://www.kernel.org/doc/html/latest/input/uinput.html>
- [4] Georgi Valkov. 2023. python-evdev: Python bindings to the Linux input handling subsystem. Retrieved from <https://python-evdev.readthedocs.io/>
- [5] OpenCV Team. 2024. Open Source Computer Vision Library. Retrieved from <https://docs.opencv.org>
- [6] The Wayland Project. 2024. Wayland Protocol. Retrieved from <https://wayland.freedesktop.org>
- [7] Charles R. Harris, K. Jarrod Millman, Stefan J. van der Walt, et al. 2020. Array programming with NumPy. Nature 585 (2020), 357–362. DOI: <https://doi.org/10.1038/s41586-020-2649-2>